# Контест 9 · Тексты, промпты и шаблоны

Неделя 18 · темы недель 17–18: текст для модели и от модели, шаблоны сообщений и проектов, сравнение промптов, проверка ответов. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

## A. Строки и токены

**A1.** `approx_tokens(text)` — грубая оценка числа токенов. Режем текст на куски: слова ( `\w+` ) и отдельные знаки — всё, что не буква, не цифра и не пробел. Кусок длины `L` стоит `ceil(L / 4)` токенов.

```
Ввод:  approx_tokens("Привет, мир!"), approx_tokens("Подсказка: ходи в центр :)")
Вывод: 5, 10
```

**A2.** `truncate(text, limit)` — обрезать для превью так, чтобы с «…» вышло не больше `limit` символов. Берём первые `limit − 1` символов; если обрезали посреди слова и пробел в куске есть — отрезаем до последнего пробела. С конца убрать пробелы и знаки `, ; : — -`, добавить «…». Влезает целиком — вернуть как есть; `limit < 2` — `ValueError`.

```
Ввод:  truncate("Сегодня мы играем в крестики-нолики", 25)
       truncate("Ход сделан, жди соперника", 14), truncate("Суперкалифрагилистик", 8)
Вывод: "Сегодня мы играем в…"
       "Ход сделан…", "Суперка…"
```

**A3.** `wrap(text, width)` — жадный перенос по словам: в строку добавляем слово через пробел, пока длина строки не больше `width`. Слово длиннее `width` сначала режем на куски по `width` символов.

```
Ввод:  wrap("Ведущий говорит: сверхдлиннословище не влезет", 10)
Вывод: ["Ведущий", "говорит:", "сверхдлинн", "ословище", "не влезет"]
```

**A4*.** `split_chunks(text, size, overlap)` — куски текста для модели. Предложения — части после разреза по пробелам, перед которыми стоит `.`, `!`, `?` или `…`. Набираем предложения в кусок (через пробел), пока его длина не больше `size`. Следующий кусок начинается с последних `overlap` предложений предыдущего (нахлёст); если с ними новое предложение не влезает — выбрасываем их по одному с начала. Предложение длиннее `size` — отдельный кусок. Зачем нахлёст, если модель видит куски по отдельности?

```
Ввод:  t = "Игра началась. Ход белых! Белые ходят e4. Чёрные думают… Ответ: e5."
       split_chunks(t, 30, 1)
Вывод: ["Игра началась. Ход белых!", "Ход белых! Белые ходят e4.",
        "Белые ходят e4. Чёрные думают…", "Чёрные думают… Ответ: e5."]
```

## B. Шаблоны

**B1.** `render(template, ctx)` — подставить `{{ имя }}` (пробелы внутри скобок необязательны). Имя может быть путём `a.b.c` : у словаря берём ключ, у объекта — атрибут. Нет значения — `KeyError(имя)`

с полным путём.

```
Ввод:  render("Привет, {{ user.name }}! Очки: {{score}}",
              {"user": {"name": "Аня"}, "score": 120})
       render("{{ user.age }}", {"user": {"name": "Аня"}})
Вывод: "Привет, Аня! Очки: 120"
       KeyError("user.age")
```

**B2.** `render_filters(template, ctx)` — как B1, плюс фильтры через `|` слева направо, аргумент после `:` (пробелы по краям аргумента убрать): `upper`; `pad:N` — выровнять вправо по ширине `N`; `escape` — HTML; `default:текст` — если значения нет, оно `None` или `""`; `plural:одна,две,пять` — число и слово в нужной форме (1, 21 — «одна»; 2–4, 22–24 — «две»; 0, 5–20, 11–14 — «пять»). Неизвестный фильтр — `ValueError`.

```
Ввод:  render_filters("{{ name|upper }} |{{ score|pad:5 }}| {{ tag|default:нет }} "
                      "{{ wins|plural:победа,победы,побед }}",
                      {"name": "bob", "score": 42, "tag": "", "wins": 22})
Вывод: "BOB |   42| нет 22 победы"
```

**B3.** `render_for(template, ctx)` — ещё и циклы `{% for x in items %}…{% endfor %}` (не вложенные). Тело рендерится через B2 для каждого элемента; внутри доступны `x` и `loop.index` (с 1), `loop.last`. Текст вне циклов — тоже через B2. Подставленные значения повторно не рендерятся.

```
Ввод:  tpl = ("Топ:\n{% for p in top %}{{ loop.index }}. {{ p.name }} — {{ p.score }}\n"
              "{% endfor %}Всего: {{ n }}")
       top = [{"name": "Аня", "score": 300}, {"name": "Боб", "score": 250}]
       render_for(tpl, {"top": top, "n": 2})
Вывод: "Топ:\n1. Аня — 300\n2. Боб — 250\nВсего: 2"
```

## C. Промпты

**C1.** `fill_prompt(template, values)` — плейсхолдеры `{имя}` как в `str.format`, `{{` и `}}` — буквальные скобки (без них не написать JSON в промпте). Ответ — `(текст, нет значения, лишние значения)`, списки по алфавиту; плейсхолдер без значения остаётся как есть.

```
Ввод:  fill_prompt('Ты — {role}. Ответь в JSON {{"hint": ...}}. Вопрос: {question}',
                   {"role": "ведущий", "lang": "ru"})
Вывод: ('Ты — ведущий. Ответь в JSON {"hint": ...}. Вопрос: {question}',
        ["question"], ["lang"])
```

**C2.** `few_shot(examples, query, k)` — промпт с примерами `(вопрос, ответ)`: выбрать `k` самых похожих на `query` по мере Жаккара множеств слов (`\w+` в нижнем регистре), при равенстве — кто раньше в списке. Блоки `"Вопрос: …\nОтвет: …"` по убыванию похожести, последний — `"Вопрос: query\nОтвет:"`, между блоками — пустая строка.

```
Ввод:  ex = [("как ходит конь", "буквой Г"), ("сколько клеток на доске", "64"),
            ("как ходит ладья", "по прямой"), ("кто ходит первым", "белые")]
       print(few_shot(ex, "как ходит ферзь", 2))
Вывод: Вопрос: как ходит конь
       Ответ: буквой Г

       Вопрос: как ходит ладья
       Ответ: по прямой

       Вопрос: как ходит ферзь
       Ответ:
```

**C3.** `extract_json(text)` — все JSON-объекты верхнего уровня из ответа модели, по порядку. Ищем `{`, считаем скобки до парной (скобки внутри строк, с учётом `\"`, не считаются) и пробуем `json.loads`. Не разобралось — ищем следующую `{` со следующего символа.

```
Ввод:  extract_json('Вот: {"a": 1} и {"b": "}{"}, {сломано}, и {"c": {"d": [1, 2]}}')
Вывод: [{"a": 1}, {"b": "}{"}, {"c": {"d": [1, 2]}}]
```

**C4*.** `diff_lines(a, b)` — построчный diff двух версий промпта через наибольшую общую подпоследовательность строк (`dp[i][j]` — НОП суффиксов). Идём с начала: строки равны — `" строка"`; иначе, если `dp[i+1][j] ≥ dp[i][j+1]`, — `"- строка из a"`, иначе `"+ строка из b"`; хвосты — `-`, затем `+`.

```
Ввод:  diff_lines("Ты — ведущий.\nОтвечай кратко.\nНе раскрывай ответ.",
                  "Ты — ведущий викторины.\nОтвечай кратко.\n"
                  "Не раскрывай ответ.\nФормат: JSON.")
Вывод: ["- Ты — ведущий.", "+ Ты — ведущий викторины.", "  Отвечай кратко.",
        "  Не раскрывай ответ.", "+ Формат: JSON."]
```

## D. Проверка ответов

**D1.** `levenshtein(a, b)` — расстояние редактирования: вставка, удаление, замена символа — по 1. Память — две строки таблицы.

```
Ввод:  levenshtein("котик", "скотина"), levenshtein("", "abc")
       levenshtein("ферзь", "ферз")
Вывод: 3, 3
       1
```

**D2.** `fuzzy_match(answer, expected, tolerance=0.2)` — ответ викторины засчитан? Нормализуем обе строки: нижний регистр, `ё → е`, знаки препинания → пробелы, лишние пробелы убрать. Засчитан, если расстояние D1 не больше `int(tolerance · длина нормализованного expected)`.

```
Ввод:  fuzzy_match("Ёлка!", "елка"), fuzzy_match("Пётр Первыи", "пётр первый")
       fuzzy_match("пётр первый", "Пётр I"), fuzzy_match("Москва", "Минск")
Вывод: True, True
       False, False
```

**D3.** `pass_at_k(n, c, k)` — модель ответила `n` раз, верных `c`. Оценка шанса, что из `k` случайных ответов хотя бы один верный: `1 − C(n−c, k) / C(n, k)` (`math.comb`), округлить до 4 знаков. Нужно `0 ≤ c ≤ n`, `1 ≤ k ≤ n`, иначе `ValueError`.

```
Ввод:  pass_at_k(10, 3, 1), pass_at_k(10, 3, 5), pass_at_k(5, 0, 2), pass_at_k(5, 4, 2)
Вывод: 0.3, 0.9167, 0.0, 1.0
```

## E. Проект и алгоритмы

**E1.** Спека расширения (`docs/ext-spec.md`, семинар 17): покажите сценарии, раздел «Не делаем» и что видит игрок, если модель или внешний сервис недоступны.

**E2.** Набор evals команды: 20+ случаев с тегами и прогон `run_evals` (лаба 16) на заглушке; объясните один случай про формат и одну атаку.

**E3.** Шаблон проекта (семинар 18): клон → `.env` из `.env.example` → `docker compose up` → бот отвечает и `/health` зелёный. Покажите, где в шаблоне нет секретов.

**E4 · босс.** Ответ модели → сообщение в Telegram с `parse_mode=HTML`. За каждую часть свой плюс.

а) `md_to_tg(text)` — Markdown модели в HTML Telegram. Сначала куски `` `код` `` → `<code>`, внутри ничего не форматируем. В остальном тексте — `html.escape(…, quote=False)`, затем: `[текст](url)` → `<a href="url">` (только `http://` и `https://`), `**жирный**` → `<b>`, `*курсив*` и `_курсив_` → `<i>`. Подчёркивания внутри слов (`my_var_name`) и `2 * 3` — не курсив.

```
Ввод:  md_to_tg("**Ход e4** — сильный. *Почему?* Смотри `a < b && c` и "
                "[правила](https://ex.org/r?a=1&b=2). my_var_name")
Вывод: "<b>Ход e4</b> — сильный. <i>Почему?</i> Смотри "
       "<code>a &lt; b &amp;&amp; c</code> и "
       '<a href="https://ex.org/r?a=1&amp;b=2">правила</a>. my_var_name'
```

б) `split_message(text, limit=4096)` — у Telegram предел длины сообщения. Пока текст длиннее `limit`: режем по последнему `"\n\n"`, при котором кусок не длиннее `limit`; нет — по `"\n"`; нет — по пробелу; нет — ровно по `limit`. Разделитель выбрасываем.

```
Ввод:  split_message("Первый абзац.\n\nВторой абзац длинный\nстрока два", 20)
       split_message("aaaaaaaaaaaaa", 5)
Вывод: ["Первый абзац.", "Второй абзац длинный", "строка два"]
       ["aaaaa", "aaaaa", "aaa"]
```

в) `sanitize(markup)` — модель прислала HTML. Оставляем теги `b i u s code pre` без атрибутов и `a` ровно с одним `href="http(s)://…"`, имя тега — в нижнем регистре. Закрывающий тег — только если он закрывает последний открытый. Всё остальное — текст: `<` и `>` → `<` `>`, `&` → `&`, если это не готовая сущность (`& < > " &#число;`). В конце закрыть открытые теги.

```
Ввод:  sanitize('<b>ok</b> <script>x</script> <i>да & <a href="https://ok.ru">тут</a>')
       sanitize("<b><i>x</b></i>")
Вывод: '<b>ok</b> &lt;script&gt;x&lt;/script&gt; '
       '<i>да &amp; <a href="https://ok.ru">тут</a></i>'
       "<b><i>x&lt;/b&gt;</i></b>"
```

**E5*.** `render_tree(files, ctx)` — генератор проекта из шаблона. `files`: путь → содержимое. Каждую часть пути рендерим через B1; часть `[if:флаг]` из пути убираем, а если `ctx.get(флаг)` ложно — файл пропускаем. Файлы `*.tmpl` рендерим через B1 и убираем `.tmpl` из имени, остальные копируем как есть. Два файла с одним путём — `ValueError`. Ответ — словарь, отсортированный по пути.

```
Ввод:  files = {"{{name}}/README.md.tmpl": "# {{ name }}",
                "{{name}}/app/main.py": "x = '{{ y }}'",
                "{{name}}/[if:bot]/bot.py.tmpl": "T = '{{ env }}'",
                "{{name}}/[if:redis]/c.py": ""}
       ctx = {"name": "snake", "bot": True, "redis": False, "env": "BOT_TOKEN"}
       render_tree(files, ctx)
Вывод: {"snake/README.md": "# snake", "snake/app/main.py": "x = '{{ y }}'",
        "snake/bot.py": "T = 'BOT_TOKEN'"}
```